Pemantauan Server dan Peringatan

Diterjemahkan dari `docs/final/server-monitoring.md` pada 1 November 2026.

- **Status:** final (disetujui 2026-11-01, D-143)
- **Tugas:** P2-08
- **Terakhir diperbarui:** 2026-11-01
- **Alat dan batas paket gratis dicek daring pada 2026-10-31** (sumber di bagian Alat).

Tujuan

Panduan ini memberi tahu pemilik, dengan cepat dan andal, saat server SIGRAK atau salah satu layanannya mati, penuh, atau gagal. Panduan ini juga memberi langkah persis untuk menyiapkannya. Cakupannya satu VPS yang dijelaskan di `server-setup.md`: IDCloudHost Jakarta, 8 vCPU / 16 GB / 320 GB, tanpa IP publik, Cloudflare Tunnel, dan staging sebagai situs kedua (D-073, D-086…D-088, D-095, D-096). Panduan ini untuk siapa pun yang menyiapkan server dan yang menanggapi peringatan.

Jawaban pemilik (2026-10-31):

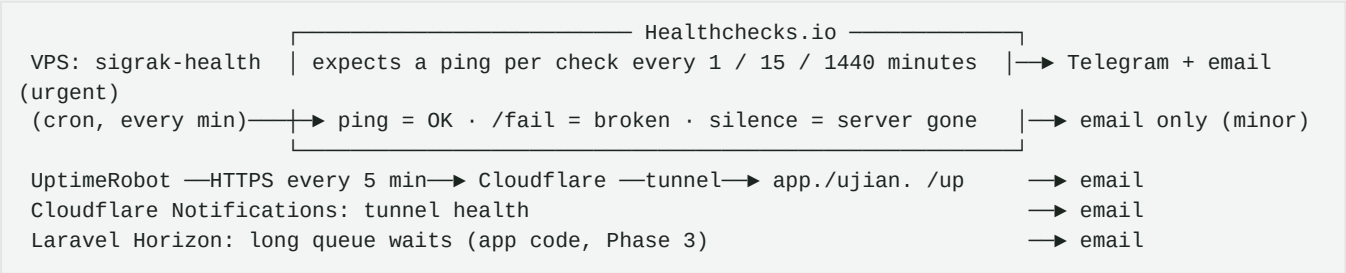
- **Saluran:** Telegram dan email.
- **Penerima:** hanya pemilik.
- **Malam hari:** peringatan mendesak boleh dikirim pada malam hari (22.00–06.00 WIB); yang ringan menunggu.

Placeholder	Arti
`<OWNER_EMAIL>`	Alamat email pemilik untuk peringatan
`<HC_API_KEY>`	API key proyek Healthchecks.io (baca-tulis). Dipakai sekali dari laptop; tidak pernah disimpan di server
`<HC_PING_KEY>`	Ping key proyek Healthchecks.io (disimpan di server)
`<HC_CH_TELEGRAM>`, `<HC_CH_EMAIL>`	UUID integrasi (saluran) Healthchecks.io
`<REDIS_PASSWORD>`, `<GOTENBERG_USER>`, `<GOTENBERG_PASSWORD>`	Nilai yang sama seperti di `server-setup.md`

Isi

1. Rancangan dalam satu gambar

Server tidak punya IP publik, jadi tidak ada pihak luar yang bisa menjangkaunya kecuali lewat tunnel. Karena itu pemantauan berjalan ke dua arah:



Ringkasan gambar: skrip `sigrak-health` di VPS berjalan lewat cron setiap menit dan mengirim ping ke Healthchecks.io (ping = baik, `/fail` = rusak, diam = server hilang). Healthchecks.io meneruskan peringatan mendesak ke Telegram + email dan peringatan ringan ke email saja. UptimeRobot memeriksa `/up` di `app.` dan `ujian.` lewat Cloudflare setiap 5 menit, Cloudflare Notifications memantau kesehatan tunnel, dan Laravel Horizon memberi tahu antrean yang menunggu terlalu lama; ketiganya lewat email.

- **Push (dead man's switch).** Sebuah skrip kecil di server memeriksa setiap komponen setiap menit dan mengirim ping ke Healthchecks.io. Bila sebuah pemeriksaan gagal dua kali berturut-turut, skrip mengirim `/fail`, dan peringatan langsung keluar. Bila seluruh VPS mati, ping berhenti, dan Healthchecks.io memberi peringatan setelah masa tenggang. Cara ini hanya memakai HTTPS keluar, yang sudah dipastikan IDCloudHost terbuka.
- **Pull (pandangan dari luar).** UptimeRobot membuka `app.` dan `ujian.` lewat Cloudflare setiap 5 menit. Ia menangkap masalah yang tidak bisa dilihat server sendiri: DNS, pengaturan Cloudflare, aturan Access yang keliru memblokir aplikasi. Ia juga memberi peringatan bila Healthchecks.io sendiri sedang mati.
- **Tingkat keparahan per saluran.** Pemeriksaan mendesak memberi peringatan lewat **Telegram dan email**. Pemeriksaan ringan hanya lewat **email**, jadi tidak pernah membangunkan siapa pun; dibaca pagi harinya. Dengan cara ini aturan "mendesak boleh malam hari, ringan menunggu" terpenuhi tanpa fitur jam tenang.

Tidak ada yang berat berjalan di VPS: satu skrip Bash, satu berkas cron, dan `curl`.

2. Alat

Dicek 2026-10-31.

Alat	Yang kita pakai	Paket gratis (saat dicek)	Bisa tanpa IP publik	Sumber
Healthchecks.io (Hobbyist)	Peringatan utama: 13 pemeriksaan, Telegram + email	20 pemeriksaan, 3 anggota tim, 100 catatan log per pemeriksaan, integrasi Telegram dan email, tanpa kartu. FAQ-nya membolehkan akun Hobbyist untuk infrastruktur perusahaan (satu akun, ≤ 20 pemeriksaan)	Ya: server yang mengirim ping keluar	healthchecks.io/pricing, /faq, /docs/api, /docs/http_api
UptimeRobot (Free)	2 monitor HTTP keyword dari luar	50 monitor, interval 5 menit, penggunaan komersial diizinkan (ketentuan diperbarui 2026-05-26). Paket gratis memberi peringatan lewat email (Telegram fitur berbayar)	Ya: ia memeriksa hostname Cloudflare yang publik	uptimerobot.com/terms, help.uptimerobot.com
Cloudflare Notifications	"Tunnel Health Alert"	Termasuk di semua paket Zero Trust, dikirim lewat email atau webhook	Ya	developers.cloudflare.com (notifikasi Tunnel)
Laravel Horizon	Notifikasi antrean yang menunggu lama (konfigurasi aplikasi)	Bagian dari Horizon; `Horizon::routeMailNotificationsTo()` dan pengaturan `waits`	Ya: mengirim email keluar	laravel.com/docs/13.x/horizon

Dipertimbangkan, tetapi belum dipilih:

- **Agen Netdata.** Agen ini bisa mengirim peringatan Telegram sendiri dan punya banyak alarm bawaan. Belum diperlukan: skrip sudah mencakup beberapa ambang yang kita pedulikan, tanpa komponen tambahan yang harus diperbarui. Ditinjau lagi bila kita butuh grafik atau riwayat.

- **Better Stack gratis.** Punya 10 monitor dan pemeriksaan 3 menit, tetapi paket gratisnya hanya memberi peringatan lewat email dan Slack (tanpa Telegram), dan halaman harganya menyebutnya "Free for personal projects".
- **Prometheus/Grafana.** Terlalu berat untuk satu VPS pada tahap ini.

Bila nanti tim butuh lebih dari 20 pemeriksaan atau ketentuan yang lebih ketat, Healthchecks.io Business berharga $20/bulan (100 pemeriksaan).

3. Yang diperiksa

Setiap komponen di `server-setup.md` punya pemeriksaan. **U** = mendesak (Telegram + email), **M** = ringan (email saja).

Slug Healthchecks	Setiap	Gagal bila	Tingkat	Catatan
`prod-app`	1 menit	`https://app.sigrak.com/up` tidak mengembalikan 200	U	Lewat Cloudflare dan tunnel, jadi menguji seluruh jalur
`prod-ujian`	1 menit	`https://ujian.sigrak.com/up` tidak mengembalikan 200	U	Prioritas tertinggi selama sesi TKA
`tunnel`	1 menit	Layanan `cloudflared` tidak aktif	U	Membedakan "tunnel mati" dari "aplikasi mati"
`host`	1 menit	Disk `/` ≥ 90 %; RAM tersedia < 1,5 GB; OOM killer berjalan dalam 2 menit terakhir	U	Pesannya menyebut penyebabnya
`mysql`	1 menit	`mysqladmin ping` gagal	U	Memakai pengguna `backup` yang hanya-baca
`redis`	1 menit	`PING` gagal; memori ≥ 90 % dari `maxmemory`; galat tulis AOF	U	Dengan `noeviction`, Redis yang penuh menghentikan antrean dan sesi
`horizon`	1 menit	`php artisan horizon:status` tidak berjalan (exit code ≠ 0)	U	Horizon ≥ 5.25 memberi exit code berbeda untuk paused dan inactive
`scheduler`	1 menit	Program Supervisor `sigrak-scheduler` tidak `RUNNING`	U	Diganti heartbeat aplikasi di Fase 3 (bagian 7)
`gotenberg`	1 menit	`GET /health` dengan basic auth tidak mengembalikan 200	U	Ekspor PDF dan kartu ujian bergantung padanya
`backup`	harian	Skrip cadangan gagal, atau tidak ada ping berhasil dalam 26 jam	U	Butuh tiga baris di skrip cadangan (bagian 6)
`host-warn`	15 menit	Disk ≥ 80 %; swap terpakai ≥ 50 %; load 15 menit ≥ 8; koneksi MySQL ≥ 160; memori Redis ≥ 70 %; ≥ 5 job gagal dalam 15 menit	M	Satu pesan mencantumkan semua peringatan
`security-updates`	harian	Pembaruan keamanan tertunda > 1 hari, atau perlu reboot	M	`unattended-upgrades` memasang sebagian besar secara otomatis
`staging`	15 menit	`/up` staging (request lokal), atau Horizon staging tidak berjalan	M	Staging tidak pernah membangunkan siapa pun

Dua kali gagal sebelum `/fail`. Satu `curl` yang gagal bisa saja gangguan sesaat. Skrip menghitung kegagalan di `/run/sigrak-health/`, dan hanya kegagalan kedua berturut-turut yang mengirim `/fail`. Satu menit yang buruk tidak mengirim apa pun, dan masa tenggang menutupinya. Gangguan sungguhan karena itu memicu peringatan dalam sekitar 2 menit.

Dari luar, oleh UptimeRobot (email):

Monitor	Jenis	Sasaran	Kata kunci
SIGRAK app	Keyword	`https://app.sigrak.com/up`	`Application up`
SIGRAK ujian	Keyword	`https://ujian.sigrak.com/up`	`Application up`

`admin.` ada di balik Cloudflare Access, jadi pemeriksa dari luar tidak bisa menjangkaunya. Ia berbagi Nginx, PHP-FPM, dan tunnel dengan `app.`, jadi `prod-app` sudah mencakupnya.

`/up` adalah rute kesehatan bawaan Laravel (diaktifkan di `bootstrap/app.php` dengan `health: '/up'`). Rute ini hanya menampilkan "Application up" tanpa data, jadi menjadi satu-satunya pengecualian teknis dari aturan "setiap halaman ada di balik login".

4. Penyiapan Healthchecks.io (sekali, dari laptop)

1. Daftar di healthchecks.io dengan `<OWNER_EMAIL>` (Hobbyist, gratis). Ganti nama proyek bawaan menjadi `SIGRAK production`.
2. **Integrations → Email:** tambahkan `<OWNER_EMAIL>`, lalu centang "up" dan "down".
3. **Integrations → Telegram:** di Telegram, buka obrolan dengan `@HealthchecksBot` dan kirim `/start`. Buka tautan konfirmasi yang dikirim bot, pilih proyeknya, lalu klik **Connect Telegram**.
4. **Settings → API keys:** buat API key baca-tulis (`<HC_API_KEY>`) dan catat **Ping key** proyek (`<HC_PING_KEY>`).
5. Cari dua UUID saluran:

   ```
   curl -fsS https://healthchecks.io/api/v3/channels/ -H "X-Api-Key: <HC_API_KEY>"
   # note the "id" of the email and the telegram entries → <HC_CH_EMAIL>, <HC_CH_TELEGRAM>
   ```

6. Buat 13 pemeriksaan. Menjalankan ini lagi akan memperbaruinya, karena `unique: ["slug"]` membuat setiap panggilan menjadi upsert:

   ```
   API='<HC_API_KEY>'
   URGENT='<HC_CH_TELEGRAM>,<HC_CH_EMAIL>'
   MINOR='<HC_CH_EMAIL>'
   mk() {  # slug timeout_s grace_s channels tags
     curl -fsS https://healthchecks.io/api/v3/checks/ -H "X-Api-Key: $API" \
       --data "
   {\"name\":\"$1\",\"slug\":\"$1\",\"timeout\":$2,\"grace\":$3,\"channels\":\"$4\",\"tags\":\"$5\"
   ,\"unique\":[\"slug\"]}" \
       -o /dev/null -w "$1 %{http_code}\n"
   }
   for s in prod-app prod-ujian tunnel host mysql redis horizon scheduler gotenberg; do
     mk "$s" 60 120 "$URGENT" "prod urgent"
   done
   mk backup           86400 7200  "$URGENT" "prod urgent"
   mk host-warn        900   900   "$MINOR"  "prod minor"
   mk security-updates 86400 43200 "$MINOR"  "prod minor"
   mk staging          900   1800  "$MINOR"  "staging"
   ```

   Setiap baris harus mencetak `201` (dibuat) atau `200` (diperbarui). Jangan simpan `<HC_API_KEY>` di server; server hanya butuh ping key.

5. Penyiapan server

Berkas pengaturan (hanya root):

```
sudo install -d -m 700 /etc/sigrak
sudo tee /etc/sigrak/monitoring.env > /dev/null <<'EOF'
HC_PING_KEY=<HC_PING_KEY>
REDIS_PASSWORD=<REDIS_PASSWORD>
GOTENBERG_USER=<GOTENBERG_USER>
GOTENBERG_PASSWORD=<GOTENBERG_PASSWORD>
EOF
sudo chmod 600 /etc/sigrak/monitoring.env
```

Skrip pemeriksaan. Skrip ini memakai ulang `/etc/sigrak/backup.cnf` (pengguna MySQL `backup` yang hanya-baca dari server-setup bagian 12):

```
sudo apt -y install update-notifier-common   # provides /usr/lib/update-notifier/apt-check
sudo tee /usr/local/sbin/sigrak-health > /dev/null <<'EOF'
#!/usr/bin/env bash
# Usage: sigrak-health minute|quarter|daily
set -uo pipefail
. /etc/sigrak/monitoring.env
HC="https://hc-ping.com/${HC_PING_KEY}"
STATE=/run/sigrak-health
mkdir -p "$STATE"
APP=/var/www/sigrak/current
STG=/var/www/sigrak-staging/current

# report <slug> <problems> [strikes]: empty problems = OK; /fail after <strikes> failures in a row
(default 2).
report() {
  local slug=$1 problems=$2 strikes=${3:-2} f="$STATE/$1.fails" n
  if [ -z "$problems" ]; then
    rm -f "$f"
    curl -fsS -m 10 --retry 2 -o /dev/null "$HC/$slug"
    return
  fi
  n=$(( $(cat "$f" 2>/dev/null || echo 0) + 1 )); echo "$n" > "$f"
  echo "$(date -Iseconds) $slug: $problems"
  [ "$n" -ge "$strikes" ] && curl -fsS -m 10 --retry 2 -o /dev/null --data-raw "$problems"
"$HC/$slug/fail"
}

# check <slug> <message> <command...>: run the command, report OK or the message.
check() {
  local slug=$1 msg=$2; shift 2
  if "$@" > /dev/null 2>&1; then report "$slug" ""; else report "$slug" "$msg"; fi
}
http_ok() { [ "$(curl -s -o /dev/null -m 15 -w '%{http_code}' "$@")" = 200 ]; }
redis() { redis-cli -a "$REDIS_PASSWORD" --no-auth-warning "$@"; }
redis_pct() {  # used_memory as % of maxmemory
  local used max
  used=$(redis INFO memory | awk -F: '/^used_memory:/{print $2+0}')
  max=$(redis CONFIG GET maxmemory | tail -1)
  [ "${max:-0}" -gt 0 ] && echo $(( used * 100 / max )) || echo 0
}
disk_pct() { df --output=pcent / | tail -1 | tr -dc '0-9'; }

minute() {
  local p
  check prod-app   "app /up not 200"       http_ok https://app.sigrak.com/up
  check prod-ujian "ujian /up not 200"     http_ok https://ujian.sigrak.com/up
  check tunnel     "cloudflared not active" systemctl is-active --quiet cloudflared
```

```
  p=""
  [ "$(disk_pct)" -ge 90 ] && p+="disk $(disk_pct)%; "
  [ "$(awk '/MemAvailable/{print int($2/1024)}' /proc/meminfo)" -lt 1536 ] && p+="RAM available < 1.5
GB; "
  journalctl -k --since "2 minutes ago" --no-pager -q 2>/dev/null | grep -qi 'out of memory' &&
p+="OOM killer ran; "
  report host "$p"

  check mysql "mysqladmin ping failed" mysqladmin --defaults-extra-file=/etc/sigrak/backup.cnf ping

  p=""
  if [ "$(redis PING 2>/dev/null)" != PONG ]; then p="no PONG"; else
    [ "$(redis_pct)" -ge 90 ] && p+="memory $(redis_pct)% of maxmemory; "
    redis INFO persistence | grep -q 'aof_last_write_status:ok' || p+="AOF write error; "
  fi
  report redis "$p"

  check horizon   "horizon not running"          sudo -u sigrak php "$APP/artisan" horizon:status
  check scheduler "sigrak-scheduler not RUNNING" sh -c 'supervisorctl status sigrak-scheduler | grep -
q RUNNING'
  check gotenberg "gotenberg /health not 200"    http_ok -u "$GOTENBERG_USER:$GOTENBERG_PASSWORD"
http://127.0.0.1:3000/health
}

quarter() {
  local p="" swap_total swap_free load conns failed
  [ "$(disk_pct)" -ge 80 ] && p+="disk $(disk_pct)%; "
  swap_total=$(awk '/SwapTotal/{print $2}' /proc/meminfo); swap_free=$(awk '/SwapFree/{print $2}'
/proc/meminfo)
  [ "$swap_total" -gt 0 ] && [ $(( (swap_total - swap_free) * 100 / swap_total )) -ge 50 ] && p+="swap
>= 50%; "
  load=$(awk '{print int($3)}' /proc/loadavg); [ "$load" -ge 8 ] && p+="load15 $load; "
  conns=$(mysql --defaults-extra-file=/etc/sigrak/backup.cnf -N -e "SHOW GLOBAL STATUS LIKE
'Threads_connected'" 2>/dev/null | awk '{print $2}')
  [ "${conns:-0}" -ge 160 ] && p+="mysql connections $conns; "
  [ "$(redis_pct)" -ge 70 ] && p+="redis memory $(redis_pct)%; "
  failed=$(mysql --defaults-extra-file=/etc/sigrak/backup.cnf -N -e \
    "SELECT COUNT(*) FROM sigrak.failed_jobs WHERE failed_at > NOW() - INTERVAL 15 MINUTE"
2>/dev/null)
  [ "${failed:-0}" -ge 5 ] && p+="$failed failed jobs in 15 min; "
  report host-warn "$p"

  p=""
  http_ok -H 'Host: staging-app.sigrak.com' http://127.0.0.1:8080/up || p+="staging /up not 200; "
  sudo -u sigrak-staging php "$STG/artisan" horizon:status > /dev/null 2>&1 || p+="staging horizon not
running; "
  report staging "$p"
}

daily() {
  local p="" sec
  sec=$(/usr/lib/update-notifier/apt-check 2>&1 | cut -d';' -f2)
  [ "${sec:-0}" -gt 0 ] && p+="$sec security updates pending; "
  [ -f /var/run/reboot-required ] && p+="reboot required; "
  report security-updates "$p" 1   # daily check: alert on the first failure
}

case "${1:-minute}" in
  minute) minute ;; quarter) quarter ;; daily) daily ;;
esac
EOF
sudo chmod 700 /usr/local/sbin/sigrak-health
```

Jalankan setiap mode sekali secara manual. Bila tidak ada keluaran, berarti semua pemeriksaan lolos. Lalu pastikan di Healthchecks.io bahwa pemeriksaannya sudah hijau:

```
sudo /usr/local/sbin/sigrak-health minute
sudo /usr/local/sbin/sigrak-health quarter
sudo /usr/local/sbin/sigrak-health daily
```

Jadwal (waktu server UTC; `daily` berjalan pukul 01:05 UTC = 08:05 WIB, jadi tiba pada siang hari):

```
sudo tee /etc/cron.d/sigrak-health > /dev/null <<'EOF'
* * * * *    root /usr/local/sbin/sigrak-health minute  >> /var/log/sigrak/health.log 2>&1
*/15 * * * * root /usr/local/sbin/sigrak-health quarter >> /var/log/sigrak/health.log 2>&1
5 1 * * *    root /usr/local/sbin/sigrak-health daily   >> /var/log/sigrak/health.log 2>&1
EOF
```

`/var/log/sigrak/health.log` hanya mencatat kegagalan. Berkas ini sudah tercakup aturan logrotate `/var/log/sigrak/*.log` (server-setup bagian 13).

Uji jalur peringatan. Hentikan Gotenberg selama tiga menit, lalu jalankan lagi. Pesan Telegram dan email seharusnya tiba dalam sekitar 2 menit, disusul pesan "up":

```
cd /opt/gotenberg && sudo docker compose stop && sleep 180 && sudo docker compose start
```

6. Heartbeat cadangan (di `server-setup.md`)

Skrip cadangan di server-setup bagian 12 sudah memuat baris-baris ini (D-136). Baris-baris itu dicantumkan di sini agar panduan ini bisa dibaca sendiri:

1. Setelah `umask 077`, skrip memuat `. /etc/sigrak/monitoring.env` dan menyetel `HC="https://hc-ping.com/${HC_PING_KEY}"`.
2. Tepat setelah itu, skrip mengirim ping mulai dan memasang trap untuk kegagalan:

   ```
   curl -fsS -m 10 --retry 2 -o /dev/null "$HC/backup/start"
   trap 'curl -fsS -m 10 --retry 2 -o /dev/null --data-raw "backup failed at line $LINENO"
   "$HC/backup/fail"' ERR
   ```

3. Tepat sebelum `echo "backup $STAMP ok"`, skrip mengirim ping berhasil:

   ```
   curl -fsS -m 10 --retry 2 -o /dev/null "$HC/backup"
   ```

Karena skrip sudah memakai `set -euo pipefail`, setiap langkah yang gagal (dump, tar, uji integritas, unggah ke R2) mengirim `/fail`. Cadangan yang tidak pernah berjalan memicu peringatan lewat diamnya ping (timeout 24 jam, tenggang 2 jam).

7. Sumber lain (dasbor dan pengaturan aplikasi)

UptimeRobot. Daftar dengan `<OWNER_EMAIL>` di paket gratis. Tambahkan dua monitor keyword dari bagian 3, dengan interval 5 menit, dan kontak peringatan email bawaan.

Cloudflare. Buka dasbor → Notifications (Observability → Alerts) → Add → **Tunnel Health Alert**, kirim ke `<OWNER_EMAIL>`. Tambahkan juga **Tunnel Creation or Deletion Event**: penghapusan yang tidak terduga adalah kejadian keamanan.

Aplikasi Laravel (Fase 3, masuk backlog):

- `Horizon::routeMailNotificationsTo(config('sigrak.alert_email'))` di `HorizonServiceProvider`, dengan `waits` diatur di `config/horizon.php`:
  - `redis:ai`: 600 dtk. Job AI wajar menunggu di belakang corong per guru dan jeda kuota.
  - `redis:exports`: 120 dtk.
  - `redis:default`: 60 dtk.

  Ini butuh penyedia email keluar milik aplikasi; Horizon tidak punya jalur Telegram.
- Heartbeat scheduler: entri jadwal setiap 5 menit dengan `->pingOnSuccess(...)` ke sebuah pemeriksaan Healthchecks.io. Heartbeat ini lalu menggantikan pemeriksaan `scheduler` yang memakai status Supervisor (slug yang sama; ubah timeout-nya menjadi 300 dtk).
- Di staging, setel `APP_ENV=staging` agar notifikasi Horizon mati di sana, atau arahkan ke email yang sama. Staging tidak pernah memakai Telegram.

8. Rute peringatan

Tingkat	Saluran	Kapan	Tanggapan
Mendesak	Telegram + email	Kapan saja, termasuk 22.00–06.00 WIB	Dilihat dalam 15 menit; saat sesi TKA, segera
Ringan	Email saja	Kapan saja (tanpa bunyi)	Pagi hari kerja berikutnya
Pandangan luar (UptimeRobot, Cloudflare)	Email	Kapan saja	Pagi yang sama, atau segera bila aplikasi juga merah di Healthchecks.io

- **Sesi ujian TKA.** `prod-ujian` diperiksa setiap menit dan memberi peringatan dalam sekitar 2 menit. Sebelum sesi terjadwal, buka dasbor Healthchecks.io dan pastikan semua pemeriksaan mendesak hijau.
- **Bisukan saat pekerjaan terencana.** Jeda pemeriksaan yang terdampak di Healthchecks.io (dasbor → pemeriksaan → Pause) sebelum deploy atau perawatan, lalu lanjutkan sesudahnya. Pemeriksaan yang dijeda aktif lagi sendiri pada ping berikutnya.
- **Hanya pemilik** yang menerima peringatan, sesuai keputusan. Untuk menambah orang kedua nanti, tambahkan sebagai anggota tim Healthchecks.io (3 orang diizinkan di paket gratis) dan tambahkan integrasi Telegram atau email kedua.

9. Runbook

Langkah pertama, berurutan, untuk setiap peringatan. SSH memakai `ssh ssh.sigrak.com`. Bila tunnel mati, pakai konsol web IDCloudHost.

Peringatan	Pemeriksaan pertama	Perbaikan biasa
`prod-app` / `prod-ujian` mati, `tunnel` hijau	1. `curl -sI -H 'Host: app.sigrak.com' http://127.0.0.1:8080/up` 2. `sudo systemctl status nginx php8.5-fpm` 3. `tail -50 /var/www/sigrak/shared/storage/logs/laravel-$(date +%F).log`	Jalankan ulang layanan yang gagal (`sudo systemctl restart php8.5-fpm` atau `nginx`). Setelah deploy yang buruk, arahkan `current` kembali ke rilis sebelumnya (server-setup bagian 10)

Peringatan	Pemeriksaan pertama	Perbaikan biasa
`tunnel` mati	1. `sudo systemctl status cloudflared` 2. `sudo journalctl -u cloudflared -n 50` 3. `sudo cloudflared tunnel info sigrak-prod`	`sudo systemctl restart cloudflared`. Bila tetap tidak tersambung: uji koneksi keluar dengan `curl -sI https://api.cloudflare.com`; bila itu juga gagal, berarti masalah jaringan IDCloudHost, jadi buka tiket
UptimeRobot mati, Healthchecks.io hijau	1. Buka situs di browser ponsel (data seluler) 2. Dasbor Cloudflare: rekaman DNS, aplikasi Access, kejadian WAF	Biasanya pengaturan Cloudflare (aturan Access yang mencakup `app.`, blokir WAF, rekaman DNS yang terhapus). Batalkan perubahan terakhir
`host` : disk ≥ 90 %	1. `df -h /` 2. `sudo du -xh --max-depth=2 /var | sort -h | tail` 3. `ls -lh /var/backups/sigrak`	Hapus rilis lama atau cadangan lokal; pastikan logrotate berjalan; `sudo journalctl --vacuum-size=300M`. Rencanakan penambahan disk bila pemakaian terus naik
`host` : RAM / OOM	1. `free -h` 2. `sudo journalctl -k --since '10 minutes ago' | grep -i oom` 3. `ps aux --sort=-rss | head`	Biasanya Chromium di Gotenberg (dibatasi 2 GB) atau terlalu banyak worker PHP. Jalankan ulang penyebabnya dan turunkan `pm.max_children` atau `maxProcesses` Horizon
`mysql` mati	1. `sudo systemctl status mysql` 2. `sudo tail -50 /var/log/mysql/error.log` 3. `df -h /`	`sudo systemctl restart mysql`. Disk penuh adalah penyebab paling umum
`redis` mati atau ≥ 90 %	1. `redis-cli -a '<REDIS_PASSWORD>' --no-auth-warning INFO memory` 2. `... --bigkeys` 3. Dasbor Horizon: job yang tertunda	Saat memori penuh, penulisan gagal (`noeviction`). Bersihkan key cache yang basi (`php artisan cache:clear` hanya membersihkan cache store bila punya database Redis sendiri, seperti usulan `architecture.md`), cari key tanpa TTL, atau naikkan `maxmemory`
`horizon` mati	1. `sudo supervisorctl status` 2. `tail -50 /var/log/sigrak/horizon.log` 3. Pemeriksaan Redis di atas	`sudo supervisorctl restart sigrak-horizon`. Bila sengaja dijeda, jalankan `php artisan horizon:continue`
`scheduler` mati	1. `sudo supervisorctl status sigrak-scheduler` 2. `tail -50 /var/log/sigrak/scheduler.log`	`sudo supervisorctl restart sigrak-scheduler`
`gotenberg` mati	1. `cd /opt/gotenberg && sudo docker compose ps` 2. `sudo docker compose logs --tail 50` 3. `free -h`	`sudo docker compose restart`. Proses yang dimatikan karena batas memori (exit 137) menandakan ekspor yang sangat besar; cari di failed jobs Horizon
`backup` gagal atau terlewat	1. `tail -50 /var/log/sigrak/backup.log` 2. `ls -lt /var/backups/sigrak | head` 3. `sudo rclone lsd offsite:<R2_BUCKET>`	Perbaiki langkah yang gagal (kredensial MySQL, disk, token R2) dan jalankan `sudo /usr/local/sbin/sigrak-backup` secara manual. Jangan pernah melewati satu hari tanpa cadangan yang baik
`host-warn`	Baca emailnya: setiap peringatan tercantum	Rencanakan perbaikan pada jam kerja. Peringatan "failed jobs" yang berulang: buka Horizon → Failed Jobs dan lihat kelas galatnya
`security-updates`	`sudo apt list --upgradable`; `cat /var/run/reboot-required.pkgs`	`sudo apt update && sudo apt -y upgrade`. Reboot di luar jam sekolah dan tidak pernah saat sesi TKA
`staging`	Langkah yang sama dengan nama `sigrak-staging`	Hanya pada jam kerja

Peringatan	Pemeriksaan pertama	Perbaikan biasa
Tunnel Cloudflare terhapus (tidak terduga)	Log audit Cloudflare	Perlakukan sebagai insiden keamanan: ganti kata sandi akun Cloudflare dan token API, lalu buat ulang tunnel

10. Letak log

Apa	Di mana	Disimpan
Kegagalan pemeriksaan kesehatan	`/var/log/sigrak/health.log`	8 minggu (logrotate)
Aplikasi Laravel	`/var/www/sigrak/shared/storage/logs/laravel-YYYY-MM-DD.log`	14 hari
Horizon, scheduler, cadangan	`/var/log/sigrak/{horizon,scheduler,backup}.log`	8 minggu
Job yang gagal	Dasbor Horizon di `admin.sigrak.com`, atau tabel `failed_jobs`	Sampai dipangkas aplikasi
Nginx	`/var/log/nginx/sigrak-{app,ujian,admin}.{access,error}.log`	Bawaan Ubuntu (14 hari)
MySQL	`/var/log/mysql/error.log`, `/var/log/mysql/slow.log`	Bawaan Ubuntu
Tunnel	`sudo journalctl -u cloudflared`	Journal, dibatasi 500 MB
Gotenberg	`cd /opt/gotenberg && sudo docker compose logs`	3 × 10 MB
Kernel / OOM	`sudo journalctl -k`	Journal
Riwayat peringatan	Healthchecks.io (100 ping per pemeriksaan), UptimeRobot, riwayat peringatan Cloudflare	Per layanan

Keputusan untuk dicatat

Dicatat di `DECISIONS.md` sebagai D-129…D-138, dengan urutan yang sama. D-143 menerima semuanya.

1. **Saluran peringatan: Telegram dan email, hanya ke pemilik; peringatan mendesak boleh dikirim malam hari** (pemilik, 2026-10-31).
2. **Healthchecks.io (Hobbyist gratis) sebagai layanan peringatan utama.** Dead man's switch berbasis push cocok untuk server tanpa IP publik. Ada 13 pemeriksaan: yang mendesak ke Telegram + email, yang ringan ke email saja. Tingkat keparahan per saluran menggantikan fitur jam tenang.
3. **Satu skrip Bash (`/usr/local/sbin/sigrak-health`) yang dijalankan cron** setiap menit, setiap 15 menit, dan setiap hari, dengan dua kegagalan berturut-turut sebelum `/fail`. Belum ada agen pemantauan (Netdata, Prometheus) di VPS.
4. **UptimeRobot gratis** untuk dua pemeriksaan keyword dari luar terhadap `/up` di `app.` dan `ujian.`, lewat email.
5. **Notifikasi Cloudflare Tunnel Health dan Tunnel Creation/Deletion** lewat email.
6. **Rute kesehatan `/up` Laravel diaktifkan.** Ini satu-satunya halaman yang bisa dibuka tanpa login, dan tidak menampilkan data.
7. **Ambang:**
   - Mendesak: disk ≥ 90 %; RAM tersedia < 1,5 GB; OOM; Redis ≥ 90 % dari `maxmemory`.
   - Ringan: disk ≥ 80 %; swap ≥ 50 %; load15 ≥ 8; koneksi MySQL ≥ 160; Redis ≥ 70 %; ≥ 5 job gagal dalam 15 menit; pembaruan keamanan tertunda atau perlu reboot.

8. **Heartbeat cadangan:** skrip cadangan mengirim ping ke `backup/start`, `backup`, dan `backup/fail` (di server-setup bagian 12, D-136).
9. **Pekerjaan aplikasi untuk Fase 3:** notifikasi email Horizon untuk antrean yang menunggu lama (`waits`: `ai` 600 dtk, `exports` 120 dtk, `default` 60 dtk) dan heartbeat scheduler dengan `pingOnSuccess`.
10. **Peringatan staging hanya lewat email**, dari satu pemeriksaan gabungan `staging`.

Pertanyaan terbuka

- [x] Paket Healthchecks.io: paket Hobbyist gratis yang dipakai (D-144).
- [x] Penyedia email: Gmail SMTP (D-110).
- [x] Penerima peringatan: hanya pemilik, tanpa penerima kedua (D-144).